DESIGN & ANALYSIS OF ALGORITHMS

Module-wise Important Topics, 4-Mark & 8-Mark Priorities

MCA • Course Code: 24MCDAAT302
Prepared from the five uploaded module notes and the October 2025 DAA previous question paper.

Your answer-length target

• **4 marks:** prepare enough material for approximately 2 handwritten pages.
• **8 marks:** prepare approximately 3–4 handwritten pages, with a definition, structured explanation, algorithm/pseudocode or diagram, worked example, and complexity/conclusion where relevant.

How to read the priorities

Very High = prioritize first; directly supported by the previous paper and/or a core algorithm. **High** = strong syllabus topic and a sensible exam preparation target. **Medium** = cover after the main topics. These are study priorities, not official probabilities or guaranteed marks.

Important: the previous paper is a useful signal, not a prediction of the next paper. Prepare at least one strong 8-mark topic from every module because the paper instructions require module coverage.

1. Exam Pattern & Strategy

The uploaded October 2025 DAA paper shows this structure: Part A has five compulsory questions carrying 4 marks each; Part B asks students to answer five full questions carrying 8 marks each, with at least one question from each module. The paper has a total of 60 marks.

Part	Pattern seen in the uploaded paper	Preparation implication
Part A	5 × 4 marks; one question from each module	Prepare a short, complete 4-mark answer for every module. Do not leave a module blank.
Part B	Answer 5 × 8 marks; at least one from each module	Prepare 1–2 dependable long-answer topics per module, then add backup topics.

A practical order of study

- **First:** study the Very High 8-mark topics listed below, especially the topics that appeared in the previous paper.
- **Second:** prepare the Very High 4-mark definitions/properties and algorithm comparisons.
- **Third:** learn one worked example or diagram for each major algorithm.
- **Finally:** revise the Medium topics and the complexity/time-bound results.

What “weightage” means in this guide

The supplied material does not provide an official topic-wise mark distribution. The guide therefore uses qualitative priority (Very High / High / Medium) based on the module notes and the topics visible in the uploaded October 2025 paper. It does not assign unsupported percentages.

2. Module 1 — Algorithm Analysis, Recurrences & Divide and Conquer

Main scope in your notes: algorithm analysis; time and space complexity; asymptotic notations; recurrence equations and their solution methods; divide and conquer; Merge Sort; Quick Sort; matrix multiplication.

Priority	Topics to cover	4-mark preparation	8-mark preparation
VERY HIGH	Asymptotic notations: O, Ω, Θ	Define each notation, meaning/bound, and give a small example.	Explain all three notations with examples and distinguish upper, lower and tight bounds.
VERY HIGH	Divide and conquer + Merge Sort	State divide/conquer/combine and outline the Merge Sort steps.	Trace Merge Sort on an array; show splitting and merging; give recurrence and time/space complexity.
VERY HIGH	Quick Sort and worst-case $O(n^2)$	Explain pivot/partition and why a poor pivot can cause worst-case behaviour.	Write/trace Quick Sort; discuss best/average/worst cases and complexity.
HIGH	Recurrence equations: substitution and iteration	Define recurrence; outline the chosen solving method.	Solve a recurrence step by step using substitution or iteration.
HIGH	Recurrence tree and Master Method	State the method and when it is used.	Build a recurrence tree or apply Master Method to a suitable recurrence.
HIGH	Time complexity and space complexity	Define both; list common growth orders O(1), O(n), O(log n), $O(n^2)$, $O(n^3)$.	Analyse a given algorithm or compare time and space complexity with examples.
MEDIUM	A priori vs a posteriori analysis; matrix multiplication	Give definitions and differences; state the matrix multiplication approach in notes.	Explain divide-and-conquer matrix multiplication and discuss its computational cost.

Previous-paper evidence: the October 2025 paper asked Quick Sort worst-case $O(n^2)$ as a 4-mark question. For 8 marks, it offered asymptotic complexity/three notations or divide and conquer in Merge Sort.

Likely questions to practise

- **4 marks:** Why is the worst-case time complexity of Quick Sort $O(n^2)$? Justify your answer.
- **4 marks:** Differentiate Big-O, Big-Ω and Big-Θ.
- **8 marks:** Explain asymptotic complexity and the three notations used to represent it.
- **8 marks:** Explain how divide and conquer is used in Merge Sort, with an example and complexity.
- **8 marks:** Solve a recurrence using the substitution/iteration method.

Study target: be able to draw a Merge Sort split/merge trace and a Quick Sort partition trace without referring to notes.

3. Module 2 — Greedy Strategy & Dynamic Programming

Main scope in your notes: greedy strategy and control abstraction; minimum spanning tree; Kruskal and Prim; fractional knapsack; job sequencing with deadlines; dynamic programming; travelling salesperson problem; Dijkstra and Bellman–Ford shortest-path algorithms.

Priority	Topics to cover	4-mark preparation	8-mark preparation
VERY HIGH	Greedy strategy + control abstraction	Define greedy choice, feasible/optimal solution, and name the control functions.	Explain the greedy strategy with control abstraction and a worked example.
VERY HIGH	Minimum Spanning Tree (MST)	Define spanning tree/MST; state that an MST on n vertices has n–1 edges.	Construct an MST step by step using Kruskal or Prim; show selected edges and total cost.
HIGH	Kruskal vs Prim	Explain the key selection process and one difference.	Run either algorithm on a weighted graph; show each chosen edge and explain cycle/connectivity handling.
HIGH	Dynamic programming	Define overlapping subproblems/optimal substructure as covered in notes; compare with greedy.	Explain a DP solution with recurrence/table and trace the result.
VERY HIGH	Travelling Salesperson Problem (TSP) using DP	Define the problem and identify the DP approach.	Solve a small TSP example using dynamic programming, showing states/costs and the final tour.
HIGH	Fractional knapsack	State the greedy selection by value/weight ratio.	Solve a numerical instance, showing ratios, item fractions, and total profit.
HIGH	Dijkstra and Bellman–Ford	State purpose and a key difference; note the limitation of Dijkstra with negative edges as described in notes.	Trace shortest-path distances; for Bellman–Ford show relaxation rounds.
MEDIUM	Job sequencing with deadlines	Define deadline/profit and outline the greedy choice.	Solve a job sequencing example with a schedule and total profit.

Previous-paper evidence: the 8-mark section included “Explain the greedy strategy with the control abstraction model” and “Explain the travelling salesperson problem using Dynamic Programming.” The 4-mark section asked for the definition of a minimum spanning tree with an example.

Likely questions to practise

- **4 marks:** Define a minimum spanning tree and illustrate it with an example.
- **4 marks:** Differentiate greedy method and dynamic programming.
- **8 marks:** Explain the greedy strategy with its control abstraction model.
- **8 marks:** Explain TSP using dynamic programming with a worked example.
- **8 marks:** Construct an MST using Kruskal’s or Prim’s algorithm.
- **8 marks:** Solve a shortest-path problem using Dijkstra or Bellman–Ford.

Study target: practise at least one weighted graph for MST, one knapsack calculation, and one DP table/recurrence.

4. Module 3 — Backtracking, State-Space Trees & Lower Bounds

Main scope in your notes: backtracking principles and terms; N-Queens/4-Queens; Sum of Subsets; state-space trees; bounding/pruning; lower-bound theory; comparison trees for sorting and searching. The previous paper also names an 8-puzzle problem using Branch and Bound.

Priority	Topics to cover	4-mark preparation	8-mark preparation
VERY HIGH	Lower-bound theory	Define lower bound and give an example.	Explain decision/comparison trees and derive a lower bound for comparison sorting or searching.
VERY HIGH	Backtracking + state-space tree	Define backtracking; explain live, dead and success nodes; state constraints.	Explain state-space trees using 4-Queens, with feasible/dead branches and a solution.
VERY HIGH	4-Queens / N-Queens	State the problem and non-attacking condition.	Show the backtracking procedure and draw/trace the state-space tree for 4-Queens.
HIGH	Sum of Subsets	Define the target-sum selection problem and role of bounding.	Trace a state-space tree for a sample set and target; show promising/non-promising branches.
HIGH	Branch and Bound / 8-puzzle	Know the problem goal and meaning of bound/state-space search.	Practise the 8-puzzle as specifically listed in the previous paper, but verify the exact algorithmic treatment in class notes.
MEDIUM	Comparison trees for sorting/searching	Define a comparison tree and its role.	Explain the decision-tree argument and lower bound using the examples in your notes.

Previous-paper evidence: 4 marks: lower-bound theory with an example. 8 marks: solving the 8-puzzle using Branch and Bound, or explaining how state-space trees are built in backtracking using the 4-Queens problem.

Likely questions to practise

- **4 marks:** Define lower-bound theory and give an example.
- **4 marks:** Explain live node, dead node and success node in backtracking.
- **8 marks:** Explain how a state-space tree is built in backtracking using the 4-Queens problem.
- **8 marks:** Explain the Sum of Subsets problem using backtracking and a state-space tree.
- **8 marks:** Explain the 8-puzzle using Branch and Bound, as asked in the previous paper.

Study target: practise drawing the tree. A clear tree with valid/dead branches and a short explanation is often the core of these answers.

5. Module 4 — Complexity Theory & Network Flows

Main scope in your notes: Class P and NP; polynomial-time reductions; NP-Hard and NP-Complete; Vertex Cover and Clique; flow networks; flow/capacity constraints; residual networks and augmenting paths; Ford–Fulkerson; Max-Flow Min-Cut theorem.

Priority	Topics to cover	4-mark preparation	8-mark preparation
VERY HIGH	Flow network properties	Define capacity constraint, flow conservation and source/sink; explain each briefly.	Explain a flow network with a labelled diagram and solve a small flow example.
VERY HIGH	Max-Flow Min-Cut theorem	State the theorem and define cut/capacity.	Explain/prove the theorem at the level in your notes, with an example network diagram.
VERY HIGH	Class P vs Class NP	Define P and NP; distinguish solving from verifying.	Compare P and NP with respect to solvability/verifiability and use examples.
HIGH	Ford–Fulkerson method	State augmenting path and residual capacity; outline the method.	Trace augmenting paths, bottleneck capacities, updated flows, and final maximum flow.
HIGH	NP-Hard vs NP-Complete	Define both; state that NP-Complete problems are in NP and NP-Hard.	Explain the conditions for NP-Completeness and the role of polynomial reduction.
HIGH	Polynomial-time reductions	Define reduction and explain why it compares problem difficulty.	Show the logic of reducing one decision problem to another, using the notes' examples.
MEDIUM-HIGH	Vertex Cover and Clique	Define each graph problem; distinguish decision and optimization versions if covered.	Explain the problem and its NP-Completeness argument from the notes.

Previous-paper evidence: 4 marks: properties of a flow network. 8 marks: P vs NP with respect to verifiability and solvability, or prove/explain Max-Flow Min-Cut with a network diagram.

Likely questions to practise

- **4 marks:** Explain the properties of a flow network.
- **8 marks:** Differentiate Class P and Class NP with respect to solvability and verifiability.
- **8 marks:** Explain/prove the Max-Flow Min-Cut theorem with an example network diagram.
- **8 marks:** Explain Ford–Fulkerson with augmenting paths and flow updates.
- **8 marks:** Explain polynomial-time reduction, NP-Hard and NP-Complete.

Study target: do one numerical maximum-flow trace and memorise the P / NP / NP-Hard / NP-Complete definitions and relationships.

6. Module 5 — Approximation & Randomized Algorithms

The Module 5 PDF is mostly visual, so the topic list is taken from its title and the readable diagrams/pages. It covers approximation algorithms; approximation ratio; the 2-approximation algorithm for Vertex Cover; randomized algorithms; basic probability; polynomial identity testing; randomized Quick Sort; and practical exposure to the Schwartz–Zippel Lemma.

Priority	Topics to cover	4-mark preparation	8-mark preparation
VERY HIGH	Significance / merits and demerits of randomized algorithms	Define randomized algorithm, mention uses, and compare it with deterministic algorithms.	Compare merits/demerits and explain when randomness helps; this appeared as an 8-mark choice in the previous paper.
VERY HIGH	Approximation ratio	Define approximation ratio and distinguish exact vs approximate solutions.	Explain ratio for minimization/maximization and use a worked example.
VERY HIGH	2-approximation for Vertex Cover	State the algorithm's main steps and the 2-approximation guarantee.	Give the step-by-step edge selection/removal algorithm, example, and show why the solution is at most twice OPT.
HIGH	Basic probability	Know sample space, event, probability and basic rules shown in notes.	Solve a probability example relevant to randomized algorithms.
HIGH	Randomized Quick Sort	Explain random pivot selection and why it is used.	Trace randomized Quick Sort and discuss expected behaviour versus worst case.
HIGH	Polynomial Identity Testing	Define the purpose and basic randomized idea.	Explain the testing approach at the level given in notes.
MEDIUM-HIGH	Schwartz–Zippel Lemma	State the practical purpose and the idea of an error bound as taught.	Explain the lemma's role in polynomial identity testing using the module's examples.

Previous-paper evidence: 4 marks: significance and uses of randomized algorithms. 8 marks: compare merits and demerits of randomized algorithms over deterministic ones in large-scale computation.

Likely questions to practise

- **4 marks:** Explain the significance of randomized algorithms and mention their uses.
- **4 marks:** Define approximation ratio.
- **8 marks:** Compare merits and demerits of randomized and deterministic algorithms.
- **8 marks:** Explain the 2-approximation algorithm for Vertex Cover with an example and guarantee.
- **8 marks:** Explain randomized Quick Sort or polynomial identity testing and the role of Schwartz–Zippel.

Study target: learn the 2-approximation algorithm as a sequence of steps and be able to explain the guarantee using the selected edges and their endpoints.

7. Previous-Paper Questions — Priority Checklist

These are the DAA questions legible in the uploaded October 2025 paper. Tick each topic only after you can write the answer without notes.

Module	4-mark question in the paper	8-mark question(s) in the paper
1	Quick Sort worst-case time complexity $O(n^2)$, with justification.	Asymptotic complexity and O/Ω/Θ; OR divide and conquer in Merge Sort.
2	Define MST with an example.	Greedy strategy with control abstraction; OR TSP using dynamic programming.
3	Define lower-bound theory with an example.	8-puzzle using Branch and Bound; OR state-space tree in backtracking using 4-Queens.
4	Explain properties of a flow network.	P vs NP with respect to verifiability vs solvability; OR Max-Flow Min-Cut theorem with a network diagram.
5	Explain the significance of randomized algorithms and mention uses.	Compare merits and demerits of randomized algorithms over deterministic ones for large-scale computation.

Use this as the first revision list. The OR alternatives matter: prepare both sides when possible, because the next paper may change the pairing.

8. How to Write Answers to Match Your Page Targets

Marks	Suggested structure	How to reach the target length meaningfully
4 marks · ~2 pages	1. Definition; 2. 4–6 key points; 3. diagram/table/example; 4. short conclusion or use.	Expand each key point into a complete sentence. For algorithms, add purpose, steps, and one small example rather than repeating the definition.
8 marks · ~3–4 pages	1. Introduction/definition; 2. principle or properties; 3. algorithm/pseudocode; 4. labelled diagram or worked example; 5. complexity/advantages/limitations; 6. conclusion.	Use a full trace/table/tree for numerical topics. Explain each step. Keep the answer structured with headings; do not pad with unrelated material.

Quick answer templates

- **Algorithm question:** definition → objective → steps/pseudocode → worked example → time/space complexity.
- **Comparison question:** define both → comparison table with 4–6 rows → suitable use/example.
- **Theorem question:** statement → explain terms → diagram → proof/logic → example/conclusion.
- **Graph/numerical question:** redraw the input clearly → show every selection/update → final result → verify the result.

Final recommendation

Start with the ten 8-mark topics shown in the previous-paper checklist, but do not study only those. The safest minimum is: one strong 8-mark answer and two 4-mark answers per module, then add a second 8-mark option in Modules 1, 2 and 4.

Source basis: DAA Module 1–5 notes and the uploaded “S3 previous year question papers” PDF. The priority labels are an informed study plan based on these materials; they are not official university weightage.